DePaul University

Jarvis College of Computing and Digital Media

School of Computing

CSC 580 Artificial Intelligence II

Dr. Noriko Tomuro

Winter 2026 - Section 801

Default Final Project: Option 1 (World Model)

Amarsaikhan Batjargal

(DePaul Student ID: 2120574)

March 20, 2026

This document was created using LaTeX on Overleaf.com

(Its LaTeX source code uploaded to Google Drive for your reference).

Collaborators:

Claude (Anthropic) for code structure guidance and debugging.

Code and Materials

Google Drive:

https://drive.google.com/drive/folders/1l2PQk-eR0uBBJ7eIqODYjViWO58VsGdp?usp=sharing

GitHub Repository:

https://github.com/amarsaikhanb/dreamer-world-model-highway

Interactive Demo (Deployed on Hugging Face):

https://huggingface.co/spaces/amarsaikhan/dreamer-highway-v0

Word Count: approximately 3000 words.

1 Introduction

This report documents the implementation of a Dreamer V1-style world model applied to the `highway-v0` environment from Gymnasium's Highway-Env package. The project represents Option 1 of the CSC 580 Default Final Project.

The idea behind world models is that instead of learning a policy through direct trial and error in the environment, we first learn a model of how the environment works. The agent can then "dream" about what would happen under various actions and use those imagined experiences to improve. The Dreamer algorithm (Hafner et al., 2020) formalized this using the Recurrent State-Space Model (RSSM), which combines a deterministic recurrent path with stochastic latent variables to capture both predictable patterns and uncertainty in the dynamics.

I implemented the full Dreamer V1 pipeline from scratch in PyTorch and applied it to highway driving. The `highway-v0` environment was configured with continuous actions (steering and acceleration), 4 lanes, and RGB visual observations. The trained agent achieves a mean reward of 119.76, compared to 14.42 for a random policy, an 8.3x improvement.

2 Background

The modern approach of learning world models from pixels was popularized by Ha and Schmidhuber (2018), who showed that a VAE combined with an RNN could learn compressed dynamics sufficient for training a controller. Dreamer V1 improved on this with the RSSM, which jointly

learns representation and dynamics. Later versions introduced discrete latents (V2) and symlog predictions (V3), but V1 was chosen here for its clarity and feasibility to implement from scratch.

Highway-Env simulates highway driving where the ego vehicle must travel fast while avoiding collisions. I configured it with continuous actions, 4 lanes, and 20 surrounding vehicles. The reward gives positive values for high speed and a negative penalty for collisions.

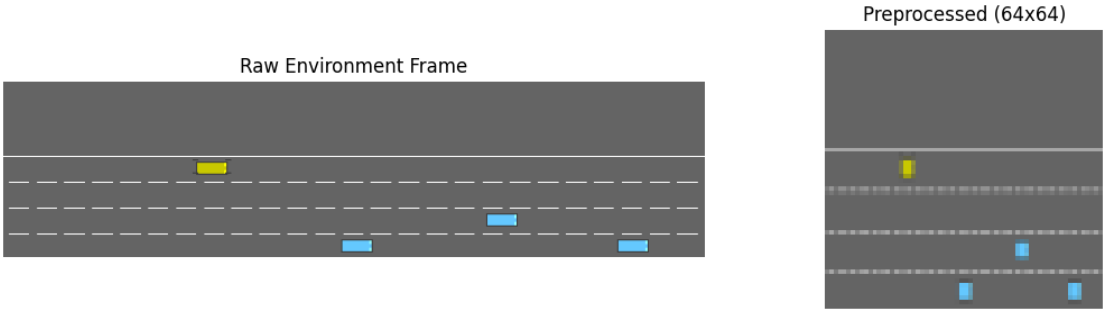

Figure 1: Left: Raw environment frame showing the 4-lane highway with the ego vehicle (yellow) and traffic (blue). Right: Preprocessed 64x64 frame used as input to the world model.

3 Method and Architecture

3.1 System Overview

The system has two main parts: the World Dynamics Model (trained with supervised learning on real transitions) and the Actor-Critic (trained with RL on imagined trajectories). Training alternates between training the world model, training the actor-critic in imagination, and collecting new episodes with the current policy.

3.2 Components

The **Encoder** is a 4-layer CNN that compresses 64x64 RGB frames into a 32-dimensional latent embedding using stride-2 convolutions. The **RSSM** maintains a 200-dimensional deterministic state (GRU) and a 32-dimensional stochastic state (Gaussian). Two networks produce the prior (for imagination) and posterior (for training) distributions. The **Reward Model** is a 3-layer MLP predicting scalar rewards from the 232-dimensional combined state. The **Decoder** uses transposed convolutions to reconstruct frames, ensuring the latent state retains visual information. The **Actor** maps latent states to Gaussian-distributed continuous actions with tanh squashing, and the **Critic** estimates expected returns using TD(λ) targets.

Table 1: Architecture summary. Total parameters: 3,087,841.

Component	**Type**	**Input → Output**	**Details**
Encoder	4-layer CNN	(3,64,64) → 32	Stride-2, depth 32–256
RSSM	GRU + MLPs	state+action → state	h=200, s=32
Reward Model	3-layer MLP	232 → 1	Hidden: 200
Decoder	4-layer DeconvCNN	232 → (3,64,64)	Mirrors encoder
Actor	2-layer MLP	232 → 2	Tanh squashing
Critic	2-layer MLP	232 → 1	TD(λ) targets

3.3 Architecture Diagram

Figure 2 illustrates the complete data flow of the Dreamer V1 system as implemented in this project. The diagram shows how an observation enters the Encoder, passes through the RSSM (which maintains both deterministic and stochastic state components), and feeds into the Decoder, Reward Model, and Actor-Critic. The RSSM contains two pathways: the posterior path (used during training, which incorporates the encoded observation) and the prior path (used during imagination, which predicts without observations). The actor's output action feeds back into the RSSM at the next timestep, forming the recurrent loop that enables multi-step imagination.

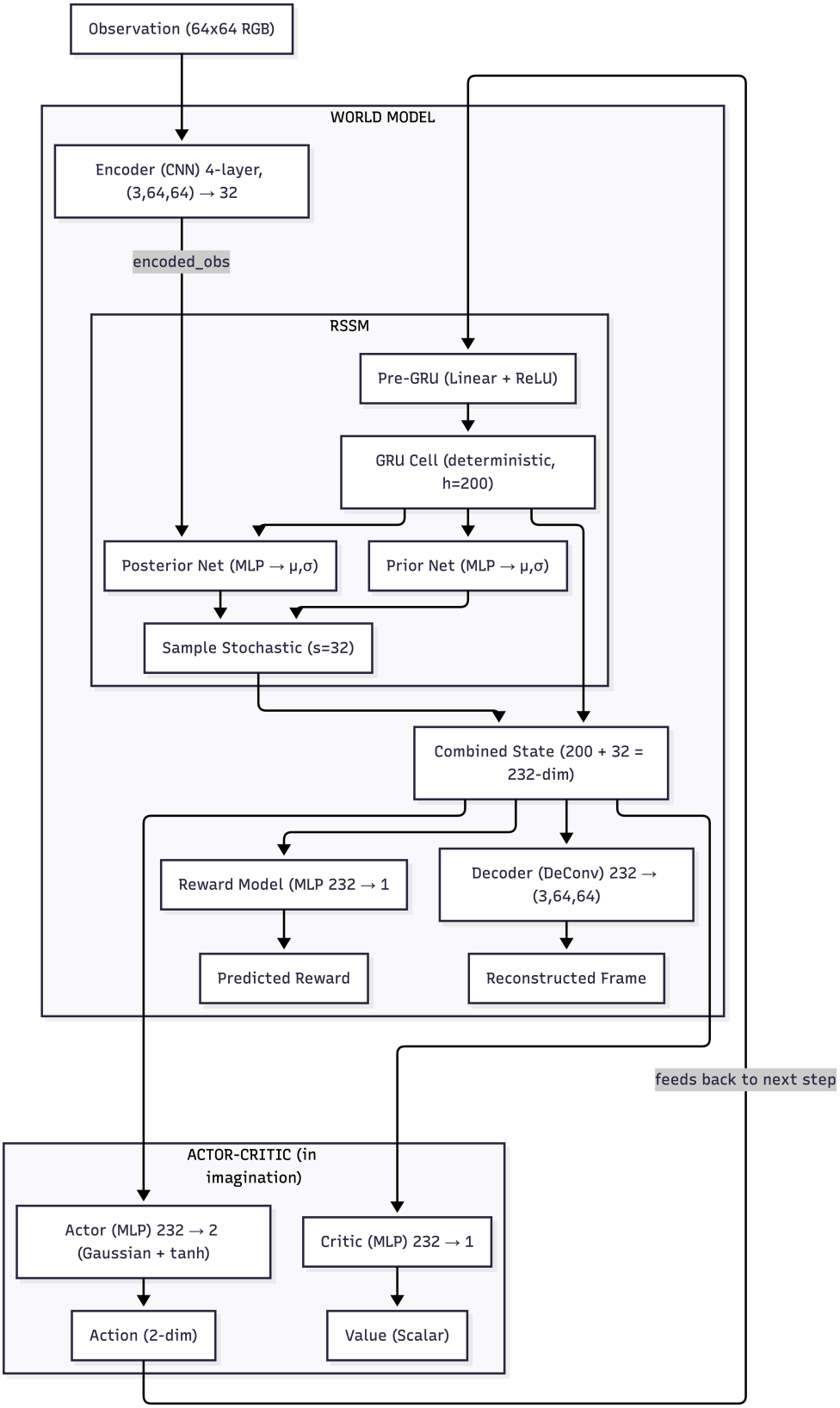

Figure 2: Architecture diagram of the Dreamer V1 world model. Observations flow through the Encoder into the RSSM, which maintains a deterministic state (GRU, h=200) and a stochastic state (Gaussian, s=32). The Prior Net predicts the next stochastic state from the deterministic state alone (used during imagination), while the Posterior Net also conditions on the encoded observation (used during training). The combined 232-dimensional state feeds into the Decoder (for frame reconstruction), Reward Model (for reward prediction), and the Actor-Critic (for policy learning). The Actor's action feeds back to the Pre-GRU at the next timestep.

The five learned models in Dreamer V1, using the notation from the original paper, are:

Table 2: Dreamer V1 learned models and their corresponding implementations.

Notation	**Name**	**Implementation**
$p_\theta(s_t \mid s_{t-1}, a_{t-1}, o_t)$	Representation model	Encoder + RSSM posterior
$q_\theta(s_t \mid s_{t-1}, a_{t-1})$	Transition model	RSSM prior
$q_\theta(r_t \mid s_t)$	Reward model	Reward MLP
$q_\phi(a_t \mid s_t)$	Action model	Actor MLP
$v_\psi(s_t)$	Value model	Critic MLP

3.4 Training Procedure

After a warm-start of 5 random episodes (about 1,000 transitions, mean reward 11.89), the main loop runs 80 iterations. Each iteration trains the world model for 100 gradient steps on 15-step sequences, trains the actor-critic for 100 steps using imagined rollouts, then collects one real episode. Critically, the actor and critic use separate imagination rollouts: the critic's is fully detached, while the actor's has gradient flow through the entire chain (actor $\rightarrow$ RSSM $\rightarrow$ reward model $\rightarrow$ targets), which is the true Dreamer V1 approach.

Key hyperparameters: FREE_NATS=0.0, KL_WEIGHT=0.1, reward loss weight=2.0, γ=0.99, λ=0.95, gradient clip=100, Adam ϵ=1e-5.

4 Results and Analysis

4.1 Training Progression

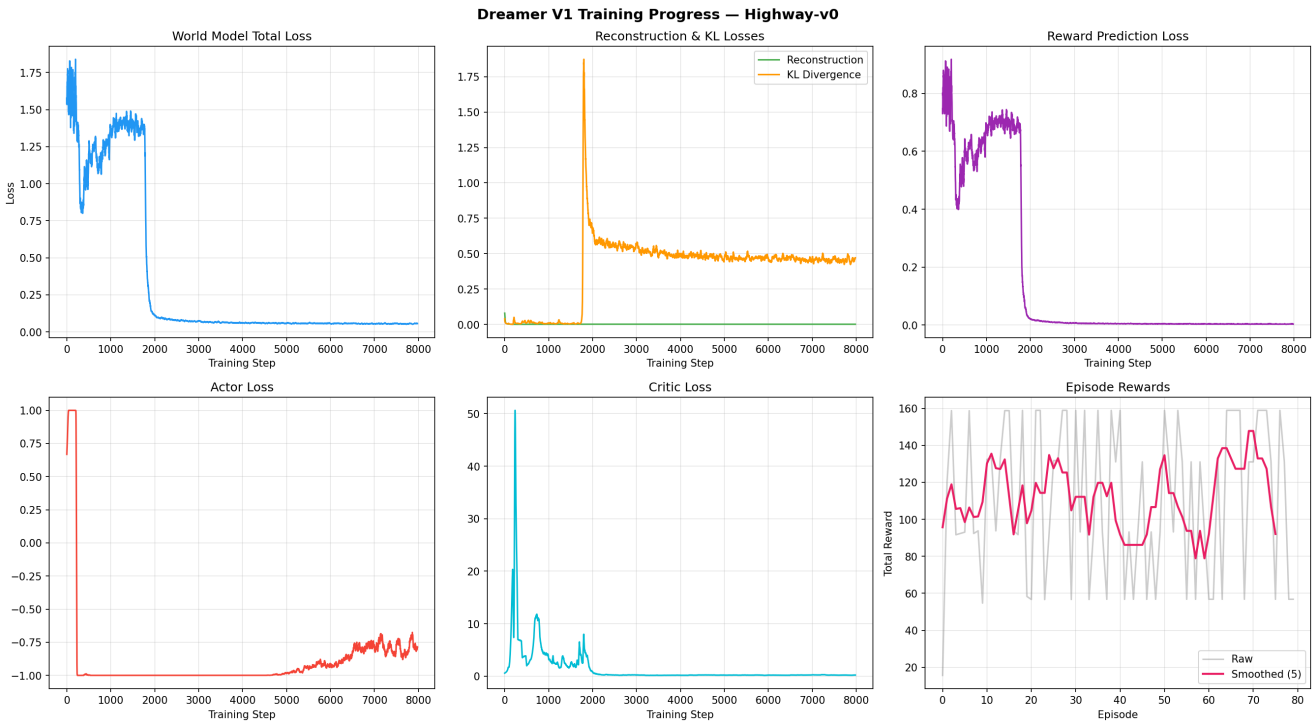

Figure 3: Training curves. Top: world model losses. Bottom: actor/critic losses and episode rewards.

The World Model Total Loss drops from 1.76 to 0.12 within the first 2,000 steps. The Reconstruction Loss converges to 0.002 almost immediately, meaning the encoder-decoder learned to compress the highway scene early. The KL Divergence spikes to 1.8 around step 1,800 (when the stochastic path activates) then settles at 0.46, a healthy level indicating the prior is reasonably accurate but the stochastic variable is being used. The Reward Prediction Loss drops from 0.9 to 0.004, a 99.5% reduction.

The Actor Loss quickly drops to -1.0 (successfully maximizing imagined returns). The Critic Loss converges from an initial spike of 50 down to about 0.2. Episode Rewards jump from 15 (random-level) to the 90–150 range within 10 episodes, peaking at 158.89.

4.2 N-Step Prediction

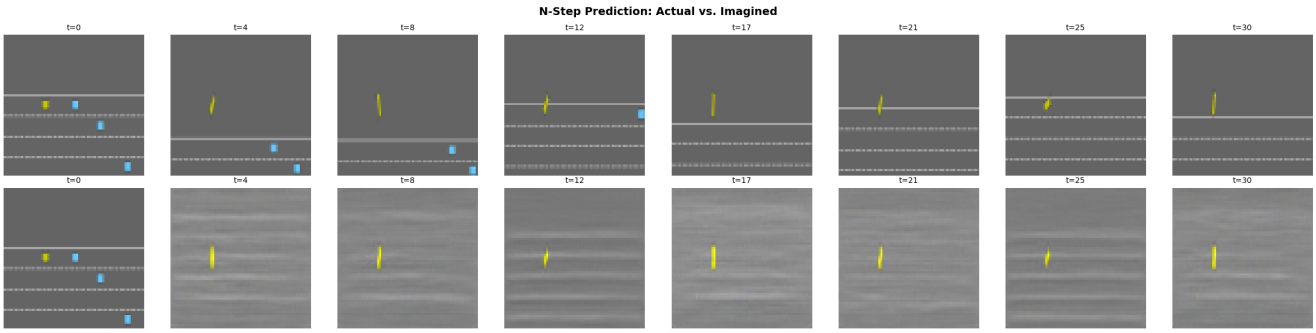

Figure 4: Actual frames (top) versus imagined frames (bottom) across a 30-step rollout. Early predictions show clear highway structure; later ones become blurry but retain the general layout.

At t=0 both rows are identical. In early timesteps (t=4, t=8), the predicted frames show the highway and ego vehicle. As the horizon extends, predictions become blurrier and vehicle positions drift, but road structure and lane positions remain visible.

4.3 Reward Prediction

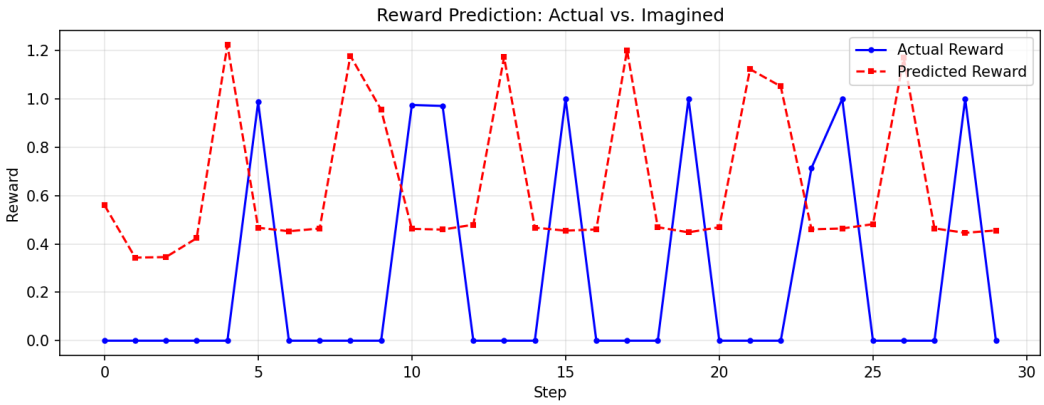

Figure 5: Actual rewards (blue) vs. predicted rewards (red dashed). The model captures the oscillating pattern but smooths over the binary-like actual signal.

Actual rewards alternate between 0 and 1.0 (low speed vs. high speed), while predicted rewards follow a smoother pattern.

4.4 Quantitative Evaluation

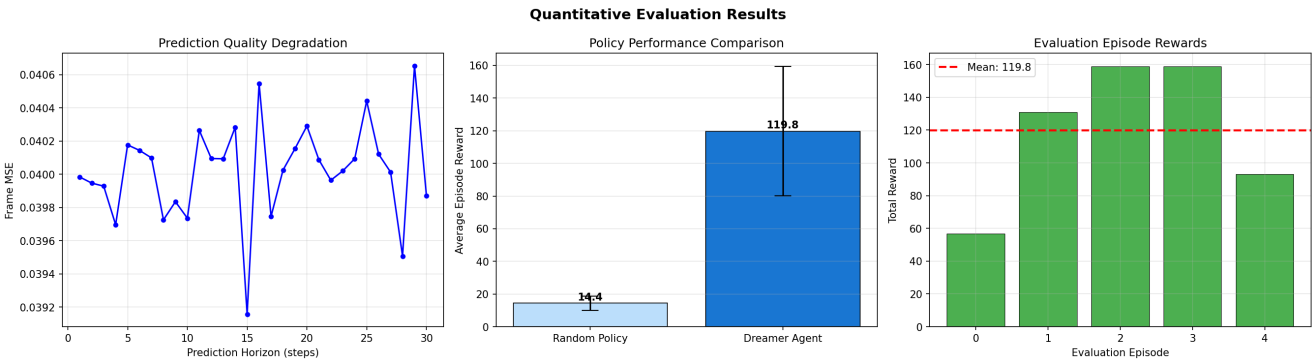

Figure 6: Left: Frame MSE stays nearly flat across the horizon. Center: Dreamer (119.8) vs. random (14.4). Right: Per-episode evaluation rewards.

Table 3: Final evaluation results.

Metric	**Value**
Random policy reward	14.42 ± 4.36
Dreamer policy reward	119.76 ± 39.65
Best episode reward	158.89
Improvement over random	8.3x
Mean episode length	200 steps (max)
1-step prediction MSE	0.0400

The agent consistently survives the full 200-step episode, indicating reliable collision avoidance. The nearly flat prediction MSE across horizons is because the highway background dominates the pixel budget; vehicle position errors barely affect overall MSE.

5 Iterative Tuning

Arriving at the final configuration required multiple rounds of debugging.

Run 1 (FREE_NATS=3.0): KL stuck at exactly 3.0 every iteration. The stochastic path was completely unused. Actor and critic losses swung wildly (3 to 346).

Run 2 (FREE_NATS=1.0, reward normalization): KL still stuck at 1.0 for 24/25 iterations. Reward loss grew instead of shrinking. Actor loss flipped positive.

Run 3 (FREE_NATS=0.0, KL_WEIGHT=0.1, reward weight 2x): Breakthrough. KL rose to 0.55, reward loss dropped 99%, rewards reached 83–140.

Run 4 (advantage normalization): Normalized advantage mean is zero by construction, so the actor gradient was exactly zero. Rewards collapsed from 100 to 6.

Run 5 (separate rollouts, target-scale normalization): Stable training, rewards consistently 90–150. This became the final configuration.

6 Discussion

The RSSM's dual deterministic-stochastic design handles the unpredictable aspects of highway traffic well. The system is sample-efficient: only 85 real episodes were needed, with imagination amplifying the data substantially. However, the 64x64 resolution is a bottleneck for fine visual details, the reward model smooths over binary reward patterns, and the reward variance across episodes (std=39.65) suggests room for improvement.

Compared to model-free approaches, the world model provides interpretable intermediate outputs (decoded frames, reward predictions) and better sample efficiency, but risks compounding prediction errors over long horizons.

One notable behavior in the agent driving videos is that the ego vehicle spins rapidly while driving forward. This occurs because the continuous action space allows large steering values (up to ± 1.0) simultaneously with high acceleration. The reward function only penalizes collisions and low speed, not erratic steering, so the agent discovers that spinning while maintaining high speed is a valid strategy that maximizes reward. Adding a steering penalty to the reward function or clipping the steering range to a smaller interval (e.g., $[-0.3, 0.3]$) would produce smoother driving, but was not pursued since the agent successfully achieves the stated objective of fast collision-free driving.

Potential improvements include discrete latents (Dreamer V2), higher resolution, a discount predictor for collision-aware value estimation, and perceptual evaluation metrics instead of raw MSE.

7 Deployment and Interactive Demo

To make the trained model accessible beyond the notebook environment, I deployed an interactive demo on Hugging Face Spaces using Gradio. The demo is publicly available at:

https://huggingface.co/spaces/amarsaikhan/dreamer-highway-v0

The demo provides three interactive modes:

1. **Actual vs Imagined**: A side-by-side comparison showing what actually happened in the environment (left) versus what the world model imagined would happen (right). Both

start from the same initial state, but the imagined side uses only the prior path (no real observations), demonstrating how prediction quality degrades over the horizon.

2. **Pure Imagination**: The world model dreams a complete trajectory from a single starting frame without any environment interaction. The actor selects actions, the RSSM predicts dynamics, and the decoder visualizes the imagined scene. This is the most direct demonstration of the world model's capability.

3. **Agent Driving**: The trained agent drives in the real highway-v0 simulation, showing the actual vehicle positions and interactions as the policy controls the ego vehicle.

Users can adjust the number of prediction steps (10 to 100) and the random seed (which changes the traffic configuration) for each mode. The demo runs on CPU and generates videos in approximately 15 to 30 seconds depending on the number of steps requested.

Deploying on a headless cloud server required a custom rendering solution since the highway-env renderer depends on pygame, which needs a display server. I implemented a lightweight renderer using OpenCV that reads vehicle positions directly from the simulation state and draws them programmatically, bypassing pygame entirely. This approach works on any platform without display dependencies.

The source code for the demo, figures, and the trained checkpoints are also available on GitHub:

https://github.com/amarsaikhanb/dreamer-world-model-highway

8 General Reflections

Building a world model from scratch was the most challenging part of this course. The individual components (CNN, GRU, MLP) are each straightforward, but getting them to work together required understanding how gradients flow through the entire pipeline. The debugging process was highly educational. For example, the free-nats threshold sounds like a minor detail but setting it too high completely disabled the stochastic path. Similarly, normalizing advantages in a way that zeroed out the gradient was a subtle but total failure mode that papers rarely discuss.

CSC 580 provided a clear progression from classical MDP, Q-Learning to world model. The concepts from earlier assignments (value functions, TD learning) appeared directly in the Dreamer architecture. Implementing simpler algorithms first gave me the foundation to understand what each Dreamer component is actually doing. I feel I've gained a solid foundation to continue working with RL systems in future research or applications.

References

1. Hafner, D., Lillicrap, T., Ba, J., & Norouzi, M. (2020). Dream to Control: Learning Behaviors by Latent Imagination. *Proceedings of the International Conference on Learning Representations (ICLR).*
2. Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., & Davidson, J. (2019). Learning Latent Dynamics for Planning from Pixels. *Proceedings of the International Conference on Machine Learning (ICML).*
3. Ha, D., & Schmidhuber, J. (2018). World Models. *arXiv preprint arXiv:1803.10122.*
4. Hafner, D., Lillicrap, T., Norouzi, M., & Ba, J. (2021). Mastering Atari with Discrete World Models. *Proceedings of the International Conference on Learning Representations (ICLR).*

End of ***Dream the Road - Tiny Dreamer***